

Classification Algorithms — Notes Explained + Working Code

A walkthrough of Logistic Regression, KNN, Naive Bayes, Decision Trees, SVM, and model-evaluation metrics, applied to one simple dataset.

1. The Dataset

To keep things concrete, every algorithm below is run on the same simple, synthetic dataset: 200 people described by **weight (kg)** and **height (cm)**, labelled **obese (1)** or **not obese (0)** — the same style of weight-vs-obesity example used in the notes. The two groups overlap on purpose, so no model gets a perfect score — just like real data.

weight_kg	height_cm	obese
72.6	156.7	0
78.3	166.6	0
79.5	158.6	0
100.2	168.2	1
114.9	157.7	1
96.4	162.2	1

2. Logistic Regression

Linear regression fits a straight best-fit line ($h(x) = \theta_0 + \theta_1 x$), which works for continuous outputs like salary, but two problems appear for a yes/no target: outliers can drag the line around, and the line's output can go below 0 or above 1 — which makes no sense for a probability. Logistic regression fixes both by squashing the straight line into an S-shaped curve using the **sigmoid function**, so the output always lands between 0 and 1.

$$g(z) = 1 / (1 + e^{-z}) \quad \text{so} \quad h\theta(x) = 1 / (1 + e^{-(\theta_0 + \theta_1 x)})$$

Here e is Euler's number (≈ 2.71828), the constant that gives the curve its smooth shape. The model is trained by finding the θ_0 , θ_1 values that best fit this squashed curve — done by minimising a cost function called **log loss** (binary cross-entropy):

$$\text{Log Loss} = -(1/m) \sum [y \log(y_{\text{pred}}) + (1-y) \log(1-y_{\text{pred}})]$$

y is the actual label (0 or 1) and y_{pred} is the predicted probability. Intuitively: if the actual label is 1 and the model predicts 0.99, the loss is tiny. If it predicts 0.01 instead, the loss is huge — a heavy penalty for a confident wrong answer. Training repeatedly adjusts θ_0 and θ_1 to slide down this loss curve toward its **global minimum**, i.e. the best-fitting θ values.

3. Evaluating a Classifier

Once a model makes yes/no predictions, a **confusion matrix** lays out how it did against the actual labels:

	Predicted: Yes	Predicted: No
Actual: Yes	TP (correct)	FN — Type II error
Actual: No	FP — Type I error	TN (correct)

- **Accuracy** = $(TP+TN) / (TP+TN+FP+FN)$ — overall % of correct predictions.
- **Precision** = $TP / (TP+FP)$ — of everything predicted positive, how much really was. Matters most when a false positive is costly (e.g. spam detection).
- **Recall** = $TP / (TP+FN)$ — of everything actually positive, how much was caught. Matters most when a false negative is costly (e.g. disease detection).
- **F1-score** = $2 \times (Precision \times Recall) / (Precision + Recall)$ — the harmonic mean of the two, used when you want a balance between them.

4. K-Nearest Neighbours (KNN)

KNN classifies a new point by looking at its **k** closest neighbours (usually measured with Euclidean distance) and taking a majority vote among them. **k** is a hyperparameter and is usually chosen as an odd number to avoid ties, and it's tuned experimentally (e.g. via cross-validation: split data into train/test, try $k = 1, 3, 5, 7 \dots$ and keep whichever gives the best accuracy). Outside classification, the same neighbour-averaging idea is used for regression by averaging the neighbours' values instead of voting.

5. Naive Bayes

Naive Bayes is built on **Bayes' theorem**, which relates the probability of A given B to the probability of B given A:

$$P(A|B) = [P(A) \times P(B|A)] / P(B)$$

For a coin toss or a single dice roll, events are **independent** — $P(A \text{ and } B) = P(A) \times P(B)$. Drawing cards from a deck without replacement is **dependent** — once one card is drawn, the probabilities for the next draw change (this is where conditional probability comes in). Naive Bayes applies Bayes' theorem to a classification problem with several features ($x_1, x_2, x_3, x_4, \dots$) by 'naively' assuming the features are independent of each other given the class, which simplifies the maths a lot and works surprisingly well in practice:

$$P(y|x_1, \dots, x_n) = [P(y) \times P(x_1|y) \times P(x_2|y) \times \dots \times P(x_n|y)] / [P(x_1) \times P(x_2) \times \dots \times P(x_n)]$$

The class ($y = 1$ or $y = 0$) with the higher resulting probability is the prediction.

6. Decision Tree

A decision tree is built from a **root**, internal **branches**, and **leaves** — plain nested if/elif/else logic, but learned from data instead of hand-written. The algorithm: (1) start with the data, (2) choose the best feature to split on, (3) create branches for each value of that feature, (4) repeat for each branch, (5) stop when a branch's data is pure or a maximum depth is reached.

Choosing the 'best' feature to split on uses **entropy** (impurity) and **information gain** (how much entropy drops after a split):

$$\text{Entropy}(S) = -p(+) \log_2(p(+)) - p(-) \log_2(p(-))$$

Information Gain(S , feature) = $\text{Entropy}(S) - \sum (|S_v| / |S|) \times \text{Entropy}(S_v)$, summed over each value v the feature can take. The feature with the highest information gain becomes the split (e.g. the root) at that step.

7. Support Vector Machine (SVM)

SVM separates two classes with a decision boundary called a **hyperplane** — a line in 2D, a plane in 3D. It is defined by:

$$w \cdot x + b = 0$$

where x is the point being classified, w is the weight/slope, and b is the bias. If $w \cdot x + b > 0$ the point falls in class 1, and if < 0 it falls in class 2. The **margin** is the gap between the hyperplane and the nearest points of each class, and the **support vectors** are exactly those nearest points — the ones that 'support' (define) the line. When classes aren't linearly separable, SVM can use a **kernel** (e.g. the RBF kernel used in the code below) to reshape the space so a straight boundary becomes possible.

8. Code: All Five Models on One Dataset

The script below builds the weight/height/obese dataset shown above, then trains and evaluates all five classifiers with scikit-learn, printing each one's confusion matrix, accuracy, precision, recall and F1-score.

```
"""
Simple Classification Demo
=====
Dataset: a small synthetic "weight & height -> obese (yes/no)" dataset,
the same kind of example used in the notes (logistic regression section).

We run FIVE classifiers on the same simple dataset:
1. Logistic Regression
2. K-Nearest Neighbours (KNN)
3. Naive Bayes
4. Decision Tree
5. Support Vector Machine (SVM)

For each model we print the confusion matrix, accuracy, precision,
recall and F1-score -- the exact metrics covered in the notes.
"""

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)

# -----
# 1. Build a simple dataset: weight (kg) & height (cm) -> obese (1/0)
#    (mirrors the weight/obesity table used in the notes)
# -----
np.random.seed(42)
n = 200

# Non-obese group: lower weight relative to height
weight_no = np.random.normal(68, 14, n // 2)
height_no = np.random.normal(165, 7, n // 2)

# Obese group: higher weight relative to height (overlapping range on purpose,
# so the models aren't perfect -- just like real data)
weight_yes = np.random.normal(85, 14, n // 2)
height_yes = np.random.normal(163, 7, n // 2)

weight = np.concatenate([weight_no, weight_yes])
height = np.concatenate([height_no, height_yes])
label = np.array([0] * (n // 2) + [1] * (n // 2)) # 1 = obese, 0 = not obese

df = pd.DataFrame({"weight_kg": weight, "height_cm": height, "obese": label})
df = df.sample(frac=1, random_state=1).reset_index(drop=True) # shuffle

print("Sample of the dataset:")
print(df.head(8).to_string(index=False))
print(f"\nTotal rows: {len(df)} | obese=1 count: {df['obese'].sum()} | obese=0 count: {(df['obese']==0).sum()}")

X = df[["weight_kg", "height_cm"]].values
y = df["obese"].values

# -----
# 2. Train / test split + feature scaling
#    (KNN and SVM are distance-based, so scaling matters)
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

```

)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# -----
# 3. Define the five models from the notes
# -----
models = {
    "Logistic Regression": LogisticRegression(),
    "KNN (k=5)": KNeighborsClassifier(n_neighbors=5),
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM (rbf kernel)": SVC(kernel="rbf"),
}

results = []

print("\n" + "=" * 60)
for name, model in models.items():
    # Distance-based models use scaled features; tree/NB use raw features
    if name in ("KNN (k=5)", "SVM (rbf kernel)", "Logistic Regression"):
        model.fit(X_train_s, y_train)
        y_pred = model.predict(X_test_s)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    cm = confusion_matrix(y_test, y_pred)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    results.append([name, acc, prec, rec, f1])

    print(f"\n{name}")
    print("-" * len(name))
    print("Confusion matrix [ [TN FP] [FN TP] ]:")
    print(cm)
    print(f"Accuracy : {acc:.3f}")
    print(f"Precision: {prec:.3f}")
    print(f"Recall   : {rec:.3f}")
    print(f"F1-score  : {f1:.3f}")

print("\n" + "=" * 60)
summary = pd.DataFrame(results, columns=["Model", "Accuracy", "Precision", "Recall", "F1"])
print("\nSummary of all models:")
print(summary.round(3).to_string(index=False))

# -----
# 4. Predict on a brand-new person
# -----
new_person = pd.DataFrame({"weight_kg": [88], "height_cm": [162]})
new_person_s = scaler.transform(new_person.values)
pred = models["Logistic Regression"].predict(new_person_s)[0]
prob = models["Logistic Regression"].predict_proba(new_person_s)[0][1]
print(f"\nNew person -> weight=88kg, height=162cm")
print(f"Logistic Regression prediction: {'Obese' if pred == 1 else 'Not obese'} "
      f"(P(obese) = {prob:.3f})")

summary.to_csv("/home/claude/classification_demo/results_summary.csv", index=False)
df.to_csv("/home/claude/classification_demo/obesity_dataset.csv", index=False)

```

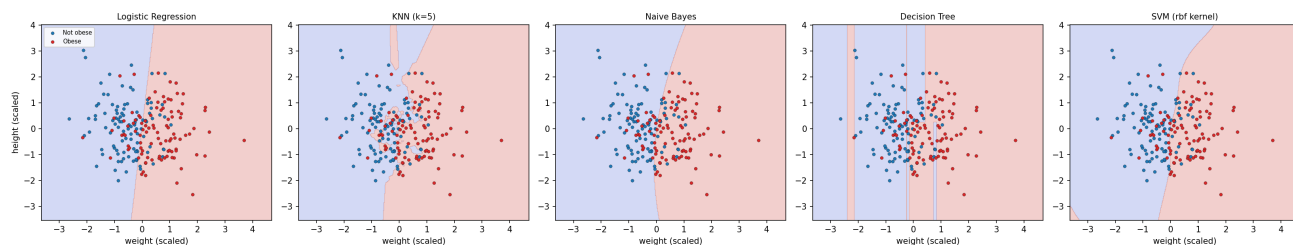
9. Results

Running the script above on the 200-row dataset (75/25 train-test split) gives:

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.740	0.731	0.760	0.745
KNN (k=5)	0.680	0.655	0.760	0.704
Naive Bayes	0.740	0.731	0.760	0.745
Decision Tree	0.700	0.692	0.720	0.706
SVM (rbf kernel)	0.700	0.708	0.680	0.694

Logistic Regression and Naive Bayes come out on top here because the true boundary between the two overlapping clusters is roughly linear/probabilistic; KNN suffers slightly more from the overlap since a few neighbours end up on the 'wrong side'. On a new person (weight = 88 kg, height = 162 cm), Logistic Regression predicts **obese**, with $P(\text{obese}) \approx 0.775$.

Decision boundaries learned by each model (same two scaled features on both axes):



Notice how Logistic Regression and SVM draw a smooth, mostly-straight boundary, KNN's boundary is jagged (it follows local neighbourhoods), and the Decision Tree's boundary is made of straight horizontal/vertical cuts (axis-aligned splits) — a direct visual of how differently each algorithm 'thinks'.